

Course: DCIT 208

Assignment number: IA2

Assignment title: Client Interview Evidence and Requirement Transformation Memo

Full name: Adwoa Nyameye Ani Asamoah

Student ID: 22412447

Team name: JASKON-Tech

Project title: ArchiVerse — A Centralized Project Tracking and Client Engagement System for Asamoah Welbeck & Associates

Team repository URL: github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-seg26-44-jaskon-tech

Individual Portfolio URL: <https://classroom.github.com/a/1-l9xPxY>

AI Disclosure: I used AI assistance. Tool: Claude (Anthropic). Purpose: structuring question set, requirement transformation template, and reflection scaffold. Output was reviewed, adapted, and verified against our team SRS (D2) before inclusion. I am able to explain and defend every part of this submission.

Date: Friday, 3rd July 2026

SECTION A

The following questions were prepared before the client interview to better understand the client's current workflow, challenges, data requirements, success criteria, security concerns, and possible system failures.

Current Workflow

How do you manage architectural projects and communicate progress to your clients?

What steps do you normally follow from the time a client contacts your company until the project is completed?

Pain Points

What are the biggest challenges you face when managing multiple projects at the same time?

Which parts of your current workflow consume the most time or cause the most frustration?

Data Requirements

What information do you collect from clients before starting a project?

What project documents or files do you need the system to store and manage?

Success Criteria

What features would make this system successful for your business?

How would you know that the new system has improved the way you manage projects?

Security and Privacy

What information should only be accessible to authorized users, and how important is protecting client data?

Failure and Exception Handling

What should happen if a client uploads an incorrect blueprint or the upload fails due to a network problem?

Additional Questions

Are there any features that you would like to add in future versions of the system?

Is there anything about your current process that you would definitely want the new system to improve or replace?

SECTION B

Date: Thursday, 2nd July 2026

Time: 10am

Medium: In-person team meeting at the Law school

Participant role: Client representative from Asamoah Welbeck & Associates; all JASKON-Tech team members present

Evidence type: Group client validation meeting

I participated in a meeting with the client from Asamoah Welbeck & Associates where I had the opportunity to ask several of my prepared discovery questions. When I asked how the firm currently manages architectural projects and communicates progress to clients, the client explained that most communication happens through email and phone calls, with project documents being shared manually. There was no central system in place, which meant that tracking project progress depended largely on individual architects remembering to update clients. When I asked about the steps followed from first client contact to project completion, the client described a largely informal process with no structured tracking tool.

On the topic of pain points, the client mentioned that managing multiple projects simultaneously was the most challenging aspect, particularly keeping clients informed about progress without it becoming time-consuming. When I asked about data requirements, the client confirmed that blueprint files, client contact information, and project status updates were the most important things the system needed to store. Regarding success criteria, the client said the system would be considered successful if clients could independently check their project status without having to call the office. For security, the client emphasized that blueprint files and client records must only be accessible to the assigned architect and client. Finally, when I raised the failure handling question about incorrect uploads or network failures, the client said the system should notify the architect immediately and allow the file to be re-uploaded without losing the project record.

SECTION C

Raw Client Statement:

"The system would be considered successful if clients could independently check their project status without having to call the office."

— Client representative, Asamoah Welbeck & Associates

1. Functional Requirement

The system shall provide a dedicated visual tracking dashboard for authenticated Clients that dynamically display the current stage, real-time updates, and active milestones of their specific architectural project.

2. Non-Functional Requirement

Availability and Usability: The project status dashboard must be fully responsive across mobile and desktop browsers and available 24 hours a day, 7 days a week, achieving a minimum uptime of 99.5% so clients can check updates independently at any time.

3. User Story

As a Client, I want to view the progress of my project on a self-service dashboard so that I can stay informed about its current status without needing to call the office.

4. Acceptance Criteria

Given an authenticated Client logs into the ArchiVerse platform, when they navigate to their account landing page, the system must display a clear, up-to-date tracking pipeline (e.g., "Design Phase", "Structural Review", "Approved") matching their project's status.

Given an Architect updates a project's phase or changes its parameters in the administrative interface, when the assigned Client refreshes or views their dashboard, the system must instantly reflect those exact updates.

Given a client has multiple projects with the firm, when they view the tracking page, the system must allow them to toggle seamlessly between each project's individual status history.

Given that a client account is deactivated or has no active projects assigned, when they access the dashboard, the system must present a neutral "No active projects found" message rather than broken or empty structural tracking fields.

5. System Design & Architectural Implications

API / Interface Implication: The backend Django REST Framework must expose a secure endpoint `GET /api/projects/{project_id}/status/` that returns a structured JSON payload containing a chronological timeline of project milestones and status flags.

Data Implication: The Project table in the PostgreSQL database must include a queryable Project Status attribute (e.g., linked to a lookup table or defined as a strict string enumeration) alongside Start Date and Expected Completion Date to track timeline progress accurately.

Test Idea: Write an automated API integration test that uses an Architect's credentials to execute a PATCH request modifying a project status from "Pending Review" to "Approved". Verify that a subsequent GET request from the assigned Client's token returns the new status with a 200 OK response code.

Open Question: Should the system automatically trigger and send email notifications to the client every time an architect updates the project status, or should updates remain silent until the client logs in?

SECTION D

Engaging in the client discovery process with Asamoah Welbeck & Associates completely redefined my understanding of how software engineering requirements dictate operational success. Before participating in this interview, I operated under the assumption that clients primarily measured the success of a project management system by its advanced, highly technical features, such as intricate CAD rendering viewing tools or automated reporting algorithms. I believed that a system's value was directly tied to the complexity of its design interfaces and administrative utilities.

However, the client's explicit success criterion—that the platform would be a success if clients could independently check their project status without calling the office—completely dismantled this assumption. This feedback made me realize that the firm's most critical bottleneck was not an engineering or design limitation, but a communication overhead issue. The real-world friction they faced lay in the hours spent managing repetitive phone calls and manually reassuring anxious clients about project milestones.

This conversation fundamentally shifted my perspective from building complex technical features to designing for self-service accessibility and immediate data visibility. It forced our team to prioritize a clean, responsive tracking dashboard over secondary, complex functionalities. I learned that effective software engineering is not about packing an application with exhaustive tools; it is about deeply understanding a client's daily operational pain points and engineering precise solutions to alleviate them. Ultimately, this interview taught me that simple, highly available data transparency can be far more impactful to a business's bottom line than complex, underutilized technical features.